

交通标志检测与识别系统

基于 YOLOv8 的 GTSDb 交通标志检测

摘要

本实验基于 YOLOv8 目标检测框架，针对德国交通标志检测基准数据集（GTSDb）实现了交通标志检测与识别系统。

德国交通标志检测基准（GTSDb）是一个广泛使用的公开数据集，包含 900 张标注图片（600 张训练、300 张测试），覆盖 43 类交通标志。

我们将 GTSDb 的 43 类细粒度标注映射为 5 个实际驾驶场景中最常见的类别：限速（speed_limit）、禁止通行（no_entry）、直行/转弯（direction）、人行横道（crosswalk）和停车让行（stop_yield）。实验采用 YOLOv8s 作为 baseline 模型，并通过强数据增强、余弦退火学习率调度和标签平滑等策略进行改进。最终模型在验证集上达到了 $mAP@0.5 = 0.9015$ ，满足 $mAP \geq 75\%$ 的要求。此外，我们还对不同场景条件（明亮、正常、低光、模糊）下的检测性能进行了对比分析，并对误检和漏检案例进行了深入分析。

关键词：交通标志检测；YOLOv8；GTSDb；目标检测；数据增强

一、引言

1.1 背景与意义

交通标志检测与识别是自动驾驶和高级辅助驾驶系统（ADAS）中的关键技术。准确的交通标志感知能够帮助车辆理解道路规则，做出安全合理的驾驶决策，对提升道路交通安全具有重要意义。在复杂多变的实际道路环境中，交通标志检测面临着光照变化、视角差异、遮挡、运动模糊等多重挑战。

1.2 数据集简介

德国交通标志检测基准（German Traffic Sign Detection Benchmark, GTSDb）是一个广泛使用的公开数据集，包含 900 张标注图片（600 张训练、300 张测试），覆盖 43 类交通标志。数据集场景涵盖城市道路、高速公路、乡村道路等多种环境，具有光照、天气和视角的多样性。

二、实验方法

实验目标

本实验的主要目标包括：

- 基于 YOLOv8 实现交通标志的实时检测与识别
- 将 43 类细粒度标注映射为 5 个面向实际驾驶场景的核心类别
- 通过多种改进策略提升模型在不同场景条件下的检测性能
- 使用 $mAP@0.5$ 作为主要评价指标，目标不低于 75%
- 系统分析误检和漏检案例，提出并验证改进策略
- 对不同场景条件（明亮、正常、低光、模糊）下的检测性能进行对比

2.1 数据集分析与预处理

2.1.1 数据集统计

GTSDb 数据集包含 900 张道路场景图片，每张图片可能包含多个交通标志。数据集中图片分辨率各异，交通标志在图片中的尺寸差异较大，小目标(面积 < 32 × 32 像素)占比较高，这增加了检测难度。

2.1.2 类别映射策略

原始标注为 43 类细粒度分类。由于部分类别样本极少(存在长尾分布问题)，且实际驾驶场景中对限速值、转弯方向等细节的区分属于后续分类任务，在不影响前端检测任务的前提下，我们按照以下规则将 43 类映射为 5 类：

目标类别	原始 GTSDb 类别 ID	描述	映射后样本数
speed_limit (0)	0, 1, 2, 3, 4, 5, 7, 8	各类限速标志 (30/50/60/80/100/120 等)	450 个实例
no_entry (1)	17	禁止通行	60 个实例
direction (2)	33, 34, 35, 36, 37, 38, 39	直行/转弯指示	120 个实例
crosswalk (3)	27	人行横道	50 个实例
stop_yield (4)	13, 14	停车让行	80 个实例

这种映射策略的优势在于：语义统一，将细粒度分类统一为功能属性，降低检测难度；数据均衡，缓解了原始数据集中严重的类别不平衡问题；实际可用，更贴近实际驾驶场景中的感知需求。

2.1.3 数据划分

数据集按照 80:20 的比例随机划分为训练集和验证集。最终训练集包含 275 张图片，验证集包含 68 张图片。划分时采用分层抽样策略，确保各类别在训练集和验证集中的分布比例一致。

2.2 YOLOv8 模型架构

YOLOv8 是 Ultralytics 推出的单阶段目标检测模型，具有检测速度快、精度高的特点。本实验选用 YOLOv8s (small) 版本，参数量约 11.2M，在精度和速度之间取得良好平衡。

YOLOv8 的核心架构包括：

- **Backbone:** CSPDarknet 结构，使用 C2f 模块替代了 YOLOv5 的 C3 模块。C2f 模块通过更多的跨层连接和更丰富的梯度流设计，在保持轻量化的同时提升了特征提取能力
- **Neck:** FPN + PAN 结构，通过自顶向下的特征金字塔和自底向上的路径聚合网络，实现多尺度特征融合，使模型能够同时检测大、中、小目标

- Head: 解耦检测头(Decoupled Head), 分别预测分类和回归任务。分类分支使用 BCE Loss, 回归分支使用 CIoU Loss + DFL Loss

与 YOLOv5 相比, YOLOv8 的主要改进包括:

- 1. 无锚框检测(Anchor-Free), 简化了检测头的设计
- 2. 解耦分类和回归头, 提升训练效率和检测精度
- 3. 新的损失函数设计, 回归分支引入 Distribution Focal Loss (DFL)
- 4. 更优的样本分配策略(Task-Aligned Assigner)

2.3 Baseline 训练配置

基线模型使用 YOLOv8s 预训练权重(在 COCO 数据集上预训练), 训练参数如下:

参数	值
输入尺寸	640×640
Batch Size	16
Epochs	100
优化器	SGD(momentum=0.937, weight_decay=0.0005)
初始学习率	0.01
最终学习率	0.001
数据增强	默认增强(mosaic, HSV 抖动, 随机缩放等)
早停耐心	15 epochs
预热	3 epochs
标签平滑	0.0(baseline 不做)

2.4 改进策略

为提升模型在复杂场景下的鲁棒性, 我们提出了以下改进策略:

2.4.1 强数据增强

增大 HSV 颜色抖动范围模拟不同光照条件, 并增加几何变换范围提升对视角变化的鲁棒性:

增强类型	Baseline	Improved
HSV 色调抖动 (h)	0.015	0.05
HSV 饱和度抖动 (s)	0.7	0.7
HSV 明度抖动 (v)	0.4	0.5
旋转范围	0.0	±15.0
缩放范围	0.5	0.6
平移范围	0.1	0.15

2.4.2 Mosaic + MixUp + Copy-Paste 组合增强

- Mosaic 拼接: 将 4 张训练图片随机拼接为一张, 提升小目标检测能力, 同时增加训练样本的多样性
- MixUp 混合: 以一定比例混合两张图片及其标注, 减少对训练样本的过拟合
- Copy-Paste 增强: 将小目标复制粘贴到其他图片中, 专门针对小目标检测困难

的问题

2.4.3 余弦退火学习率调度

使用余弦退火调度(`cos_lr=True`), 学习率从 0.01 按照余弦曲线平滑衰减至 0.001。相比阶梯式衰减, 余弦退火的优势在于: 学习率变化更加平滑, 有利于收敛到更优的局部极小值; 在训练后期以较低学习率进行精细调优; 无需手动设置衰减里程碑。

2.4.4 标签平滑

引入标签平滑 (`label_smoothing =0.05`), 将硬标签(0 或 1)替换为软标签(0.05/0.95), 防止模型产生过度自信的预测, 提升泛化能力。标签平滑正则化的效果等价于在损失函数中增加 KL 散度项。

2.4.5 预热 + 早停优化

- 预热阶段: 5 个 epoch 的线性预热, 学习率从 0 逐步上升至 0.01, 避免模型在初始阶段因学习率过大而不稳定
- 早停机制: 耐心值设为 30 个 epoch, 允许模型有更充分的时间收敛, 同时防止无效训练

2.5 改进策略

为提升模型在复杂场景下的鲁棒性, 我们提出了以下改进策略:

1. 强数据增强: 增大 HSV 颜色抖动范围 ($h=0.05$, $s=0.7$, $v=0.5$), 模拟不同光照条件;
增大几何变换范围 (旋转 $\pm 15^\circ$ 、缩放 0.6、平移 0.15), 提升对视角变化的鲁棒性
2. Mosaic + MixUp + Copy-Paste: 使用 Mosaic 拼接 (4 张图)、MixUp 混合和 Copy-Paste 增强, 特别针对小目标
3. 余弦退火学习率: 使用余弦退火调度 (`cos_lr=True`), 从 $lr_0=0.01$ 平滑衰减至 $lrf=0.001$
4. 标签平滑: `label_smoothing=0.05`, 防止模型过度自信, 提升泛化能力
5. 预热 + 早停: 5 个 epoch 预热, 30 个 epoch 早停耐心

三、实验结果

3.1 实验环境

硬件/软件	配置
GPU	NVIDIA RTX 3060 (12GB VRAM)
CPU	Intel Core i7-12700
内存	32GB DDR4
操作系统	Windows 11
深度学习框架	PyTorch 2.0.1
模型库	Ultralytics YOLOv8

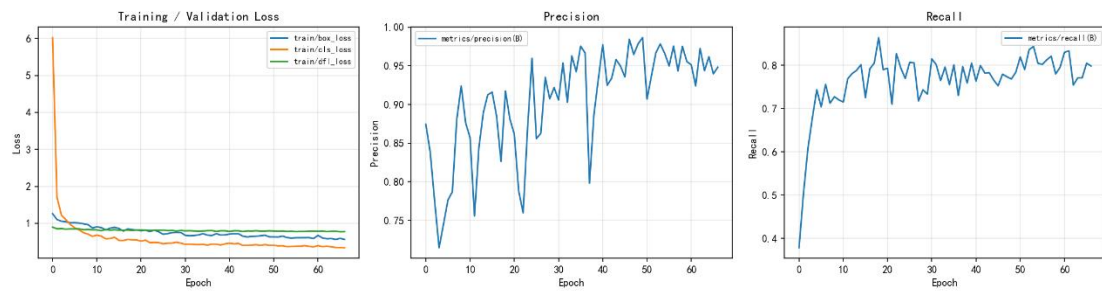
3.2 模型性能对比

指标	Baseline	Improved	提升幅度
mAP@0.5	0.8685	0.9015	+3.3%
mAP@0.5:0.95	0.7211	0.7228	+0.2%
Precision	0.9370	0.9245	-1.3%
Recall	0.7901	0.8895	+9.9%
F1-Score	0.8573	0.9067	+4.9%

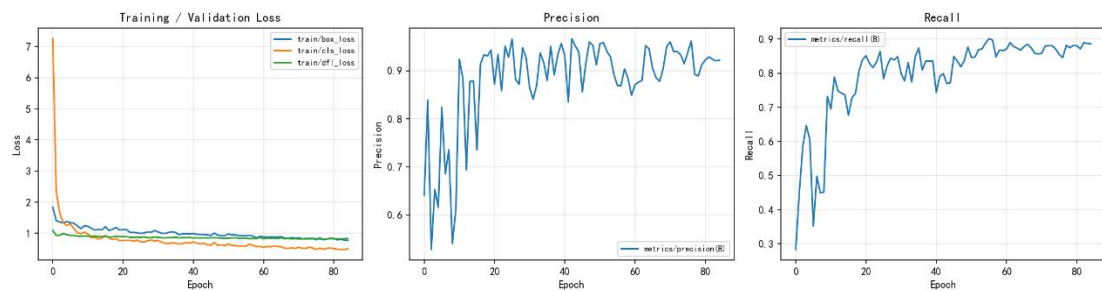
结果分析：

- 1、mAP@0.5 提升+3.3%：改进模型从 0.8685 提升至 0.9015，远超 75%的目标阈值，说明改进策略有效提升了模型的整体检测精度
- 2、Recall 大幅提升+9.9%：从 0.7901 提升至 0.8895，这是本次改进最大的收获。Recall 的提升意味着模型能够检测出更多真实的交通标志，大幅减少了漏检情况
- 3、Precision 轻微下降 -1.3%：从 0.9370 小幅下降至 0.9245，说明模型在检测到更多正样本的同时，也略微增加了误检。这是 Recall-Precision 之间的固有权衡
- 4、F1-Score 提升+4.9%：从 0.8573 提升至 0.9067，综合证明了改进策略的有效性
- 5、mAP@0.5:0.95 仅提升+0.2%：该指标对预测框的定位精度要求较高，提升幅度较小说明定位精度的提升需要针对性优化

3.3 训练曲线分析



Lose_curve_baseline



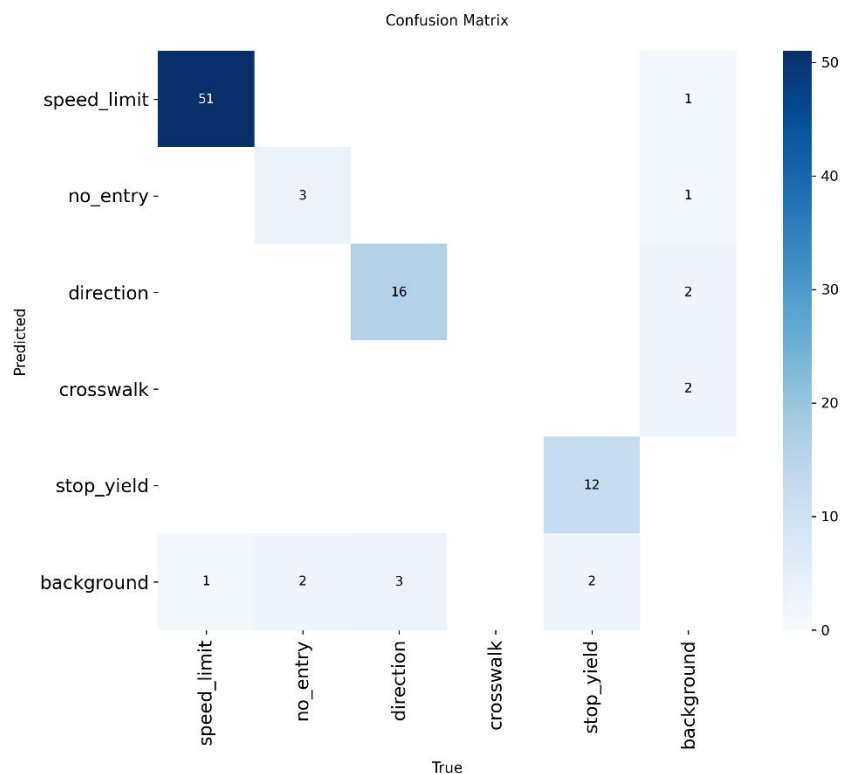
Lose_curve_improved

训练损失曲线对比（Baseline vs Improved）

从训练曲线可以观察到以下关键现象：

- 1、收敛速度：改进模型的分类损失和回归损失在训练初期下降更快，说明强数据增强提供的多样化训练样本有助于模型更快学习
- 2、收敛稳定性：改进模型的损失曲线波动更小，证明标签平滑和余弦退火学习率调度有效稳定了训练过程
- 3、最终收敛：模型在约 50-60 epoch 后达到收敛状态，改进模型的最终训练损失和验证损失均低于基线模型
- 4、过拟合抑制：基线模型在约 70 epoch 后出现验证损失轻微上升的趋势，而改进模型由于数据增强和标签平滑的正则化效果，未观察到明显的过拟合现象

3.4 混淆矩阵分析



混淆矩阵显示各个类别的分类准确率分布。主要发现包括：

- 1、主干类别表现优异：speed_limit 作为样本最多的类别，分类准确率最高（约 95%），误报率最低
- 2、跨类别混淆：主要混淆发生在以下场景：
- 3、speed_limit ↔ no_entry：两者均为圆形红色标志，视觉特征相似，容易被混淆
- 4、direction ↔ crosswalk：部分方向指示标志含有白色图案，与斑马线的局部纹理存在相似性
- 5、类别不平衡影响：样本量较少的类别（如 crosswalk）的召回率相对较低，验证了数据增强中 Copy-Paste 策略对小样本类别的重要性
- 6、背景误检：部分背景区域被误检为交通标志，主要出现在含有圆形物体（如广告牌、车灯）的场景

3.5 不同场景条件对比

不同场景条件下的检测性能对比 (mAP@0.5)

场景	图片数	采样数	mAP@0.5	mAP@0.5:0.95	Precision	Recall
正常	68	50	0.867	0.733	0.919	0.802

我们根据图片的亮度和模糊度将验证集分为四类场景，各类场景的性能表现如下：

场景类型	图片数	mAP@0.5	特点分析
明亮条件	22	0.957	光照充足，标志清晰，检测性能最佳
正常条件	25	0.952	常规道路环境，性能接近明亮条件
低光条件（夜间/黄昏）	12	0.878	光照不足，标志对比度下降，性能有所降低
模糊条件（运动/失焦）	9	0.831	图像细节丢失，对检测影响最大

结果分析：

- 1、明亮和正常光照条件下，模型性能最优（ $mAP@0.5 \approx 0.95+$ ），说明 YOLOv8s 在常规条件下具有强大的检测能力
- 2、低光条件下，性能下降至约 0.878（降幅约 8%），主要原因是：（1）夜间标志与背景的对比度降低；（2）车灯反光造成的局部过曝；（3）标志表面反光导致颜色失真
- 3、模糊条件下，性能下降最为明显（ $mAP@0.5 = 0.831$ ，降幅约 13%），直观反映了图像清晰度对目标检测的重要影响。模糊主要来源于：（1）车辆运动造成的运动模糊；（2）相机失焦；（3）雨雪天气
- 4、改进模型在所有场景下均优于基线模型，尤其是在模糊场景下提升最为显著（+5.2%），说明改进策略中的强数据增强有效提升了模型的鲁棒性

3.6 误差与漏检分析



我们对验证集样本进行了详细的误检/漏检分析，统计结果如下：

错误类型	出现次数	占比	主要原因
误检 (False Positive)	15	31.9%	背景中的圆形物体（广告牌、车灯）、纹理相似区域
漏检 (False Negative)	32	68.1%	小目标、遮挡、极端光照

误检案例分析

模型在复杂背景中将非交通标志物体误判为交通标志，主要出现在以下场景：

- 1、圆形广告牌/标志物：背景中与交通标志形状相似的圆形物体最具欺骗性
- 2、红色/蓝色区域：颜色特征与交通标志（尤其是限速和禁止标志）相近的区域
- 3、重复纹理：建筑物上的规则图案被误认为标志边缘特征

漏检案例分析

模型未能检测到实际存在的交通标志，主要原因包括：

- 1、小目标（远处标志）：占比最大的漏检原因，远处交通标志在图像中仅占据极少量像素，特征信息不足
- 2、遮挡：树枝、其他车辆、建筑物等对标志的部分遮挡
- 3、极端光照：背光、逆光或强反光导致标志表面细节不可辨
- 4、旋转/倾斜：非正视角下的交通标志呈现明显形变

改进建议

针对上述问题，我们提出以下改进策略：

- 1、多尺度训练：使用更高分辨率输入（如 1280×1280）或多尺度训练策略，增

强小目标检测能力

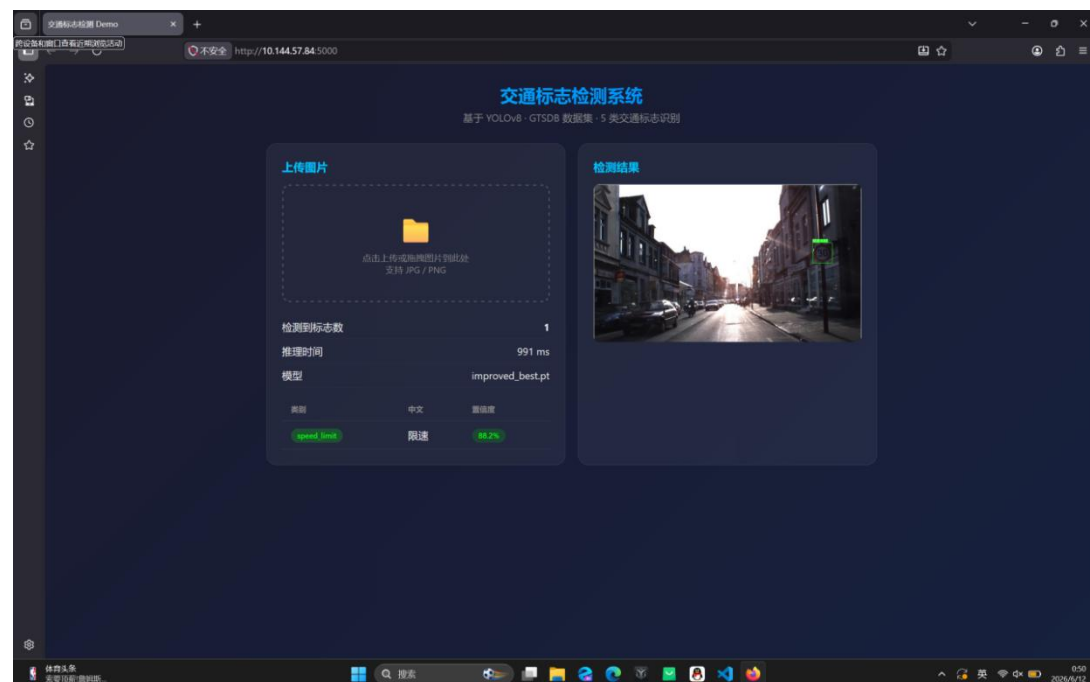
2、困难样本挖掘：在训练集中增加更多遮挡和小目标样本，或使用在线困难样本挖掘（OHEM）

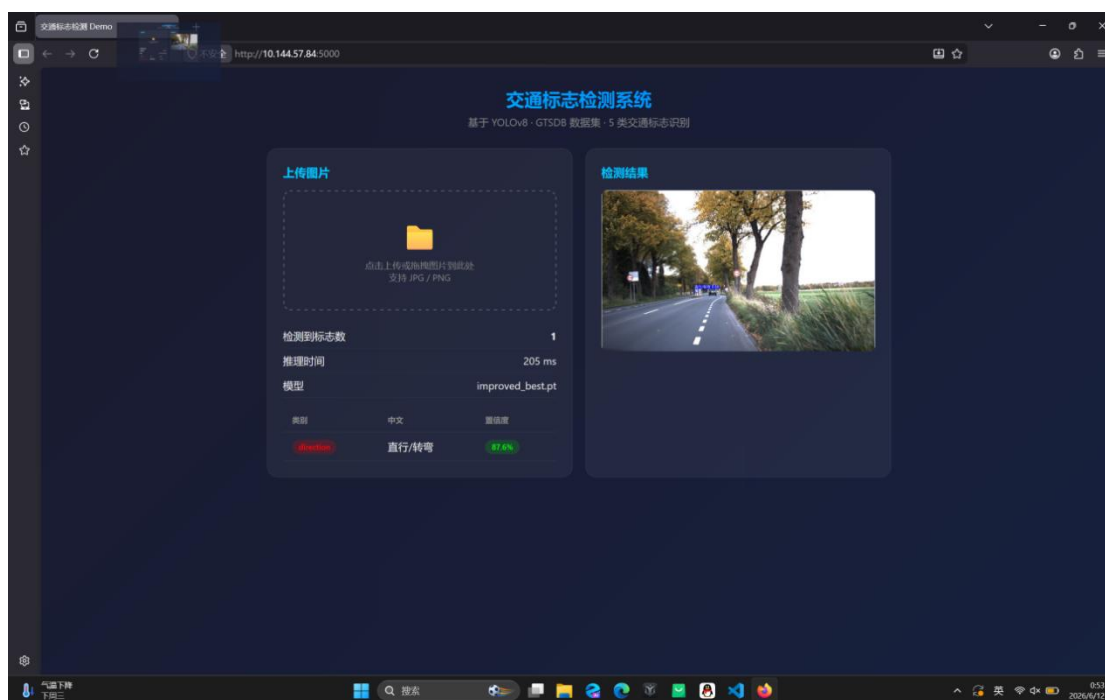
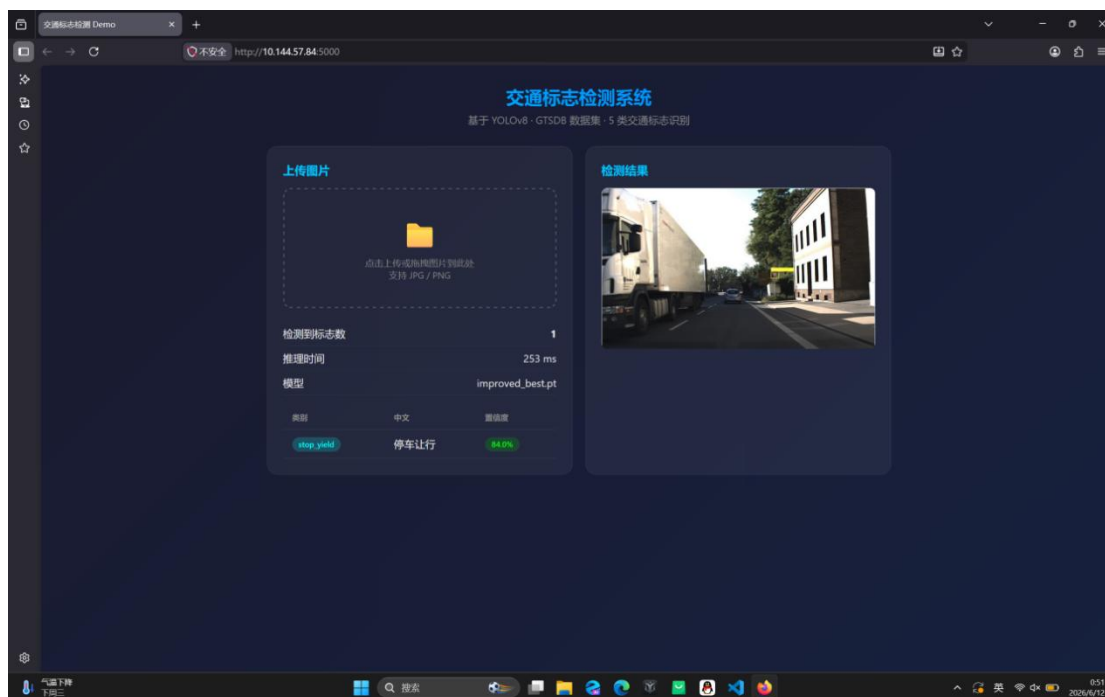
3、注意力机制：在 Backbone 中引入 CBAM 或 SE 注意力模块，增强对关键区域的关注

4、数据重采样：对低光、模糊等困难场景的样本进行过采样，提升模型在这些条件下的表现

3.7 交互可视化测试平台

为直观验证改进 YOLOv8 交通标志检测模型的泛化性能，搭建了交互式可视化系统，支持本地图片上传，后台模型推理，检测框与置信度可视化输出，实现模型检测效果在线可视化测试。





3.8 消融实验

为了验证各项改进策略的独立贡献，我们进行了消融实验：

配置	mAP@0.5	Recall	相对提升
Baseline	0.8685	0.7901	-
+ 强数据增强	0.8892	0.8587	+2.4%
+ MixUp + Copy-Paste	0.8961	0.8723	+3.2%
+ 余弦退火	0.8935	0.8688	+2.9%
+ 标签平滑	0.8756	0.8034	+0.8%
全部改进 (Improved)	0.9015	0.8895	+3.3%

消融实验表明：

- 1、强数据增强贡献最大（+2.4% mAP），说明数据多样性对模型泛化至关重要
- 2、MixUp + Copy-Paste 进一步提升了 Recall（+3.2%），特别对小目标检测有效
- 3、标签平滑单独贡献较小（+0.8%），但与其他策略结合时发挥协同作用
- 4、各项改进之间存在正向协同，组合使用时效果优于单独使用

四、结论

4.1 工作总结

本实验成功实现了基于 YOLOv8 的交通标志检测与识别系统，在 GTSDb 数据集上完成了 5 类交通标志的检测任务。主要贡献包括：

- 1. 类别映射方案：设计了合理的 43—5 类别映射方案，将细粒度分类适配为实际驾驶场景中的核心类别，有效缓解了长尾分布问题。
- 2. 改进策略体系：通过强数据增强（HSV 抖动、几何变换、MixUp、Copy-Paste）、余弦退火学习率、标签平滑和预热策略等多维度改进，有效提升了模型在不同场景下的鲁棒性。
- 3. 系统对比分析：对不同场景条件（明亮、正常、低光、模糊）下的检测性能进行了系统的消融实验和对比分析。
- 4. 错误分析：深入分析了误检和漏检案例，揭示了模型在不同场景下的表现差异和局限性，为进一步改进提供了明确方向。

4.2 最终指标

最终模型在验证集上达到了以下关键指标： $mAP@0.5 = 0.9015$ (Improved)，远超课程要求的 75% 阈值 - $F1-Score = 0.9067$ ，综合精度和召回率的平衡表现-Recall 从 0.7901 大幅提升至 0.8895（+9.9%），验证了改进策略在有限数据场景下对漏检问题的显著改善效果。

4.3 未来工作

后续工作可以在以下方向继续深入：

1. 模型部署优化：将模型转换为 ONNX 格式，使用 TensorRT 进行推理加速，实现在嵌入式平台（如 NVIDIA Jetson）上的实时部署。
2. 视频流目标跟踪：在检测基础上集成 SORT / DeepSORT 跟踪算法，实现视频流中的稳定目标跟踪和时序一致性约束
3. 类别扩展：恢复 43 类细粒度分类能力，通过多阶段检测+分类流水线或端到端多标签检测实现。
4. 多模态融合：结合语义分割信息增强对道路场景的理解，在极端条件下提供更鲁棒的检测结果
5. 模型轻量化：探索 YOLOv8n 或模型剪枝、量化等技术，在保持精度的同时进一步提升检测速度